

HyNoC: A Hybrid Circuit-Switch/Wormhole Network-on-Chip for Distributed VLIW Computing on FPGA

Christophe Clienti
Open Hardware Project
CERN-OHL-P v2.0

<https://github.com/cclienti/verilog-ip>

Abstract—Network-on-Chip (NoC) architectures have become the standard interconnect fabric for many-core systems, yet most proposals face a fundamental trade-off between latency, area, and congestion management. This paper presents HyNoC (High-performance Network-on-Chip), an open-source NoC architecture that combines circuit-switch path establishment with wormhole data transfer, targeting distributed computing systems built around VLIW processor cores on FPGA.

HyNoC employs source routing, where the complete path through the network is encoded in the packet header by the sender or statically at compile time, enabling both deterministic low-latency transfers and software-level hotspot avoidance without the area overhead of virtual channels. The router features a parallel round-robin arbiter (PRRA) with fixed grant latency, per-port independent clock domains, and support for both unicast and multicast routing.

We discuss the design rationale, the hybrid switching model, and positioning relative to prior NoC art, and argue that a richer topology combined with compiler-assisted static routing is a competitive alternative to virtual channels for FPGA-based distributed VLIW systems. The complete RTL implementation is released as open-source hardware under the CERN-OHL-P v2 license.

Index Terms—Network-on-Chip, FPGA, source routing, wormhole switching, circuit switching, VLIW, parallel round-robin arbiter, open-source hardware.

I. INTRODUCTION

As the number of processing elements integrated on a single chip increases, scalable on-chip communication becomes a first-class design concern. Network-on-Chip (NoC) architectures have emerged as the standard answer to this challenge, replacing shared buses and point-to-point links with a structured, packet-switched interconnect fabric [1].

Existing NoC designs span a broad design space. Wormhole switching minimizes buffering by forwarding flits as soon as the header is decoded, but exposes the network to head-of-line blocking. Virtual channels (VCs) were introduced to mitigate this blocking [2], at the cost of duplicated buffers that can represent the dominant area contribution of a router [3]. Circuit switching provides fully dedicated, contention-free paths but requires path reservation before any data can flow.

HyNoC takes a different approach: it combines the path-dedication property of circuit switching—through a source-routed request/grant handshake at each hop—with the flit-level, FIFO-backed flow control of wormhole switching. This hybrid

model is particularly well suited to FPGA targets, where block RAMs used as FIFOs are scarce resources, and to distributed VLIW computing workloads, where communication patterns are largely known at compile time and can therefore be routed statically to avoid congestion.

Originally inspired by the Hermes NoC [4], HyNoC departs significantly from its ancestor by introducing distributed arbitration, relative source routing with multicast support, per-port asynchronous clock domains, and a parallel round-robin arbiter that grants requests in a bounded number of cycles.

The contributions of this paper are:

- A precise characterization of HyNoC’s hybrid switching model and its implications for latency and area;
- A discussion of static route management as an alternative to virtual channels, supported by a combinatorial analysis of shortest-path counts in mesh and higher-dimensional topologies;
- An analysis of the scope and limitations of the no-VC design choice;
- An open-source RTL implementation targeting both simulation and FPGA synthesis.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the HyNoC architecture. Section IV details the hybrid switching model and the rationale for avoiding virtual channels. Section V discusses the target application domain. Section VI presents implementation details, and Section VII concludes.

II. RELATED WORK

A. Switching Strategies

Three main switching strategies are found in the NoC literature. *Store-and-forward* switching requires a complete packet to be buffered at each router before forwarding, providing clean error isolation but imposing large buffer requirements and high latency [1]. *Wormhole switching* [5] pipelines flits across routers, dramatically reducing buffer depth requirements; latency depends on path length and contention. *Circuit switching* [6] reserves a dedicated path before transmission, eliminating mid-network contention at the cost of reservation overhead.

Hybrid approaches have been explored. *Æthereal* [7] combines guaranteed-throughput circuit-switched slots with best-effort wormhole traffic on the same physical links. HyNoC differs: it uses source routing to establish a dedicated path through a series of request/grant handshakes (the circuit phase), then streams flits in wormhole fashion along that path (the data phase), with no time-division multiplexing.

B. Routing Algorithms

XY deterministic routing [1] is simple and deadlock-free on 2D meshes but provides only a single path between any pair of nodes. Adaptive routing [8] improves load balancing but requires more complex router logic and typically needs virtual channels to avoid deadlock.

Source routing encodes the complete route in the packet header at the sender. This approach, studied for NoCs in [9] and [10], shifts routing complexity from the router to the sending node or to an offline compilation step. The router itself becomes simpler: it only needs to decode the next hop from the current flit, forward it, and update the index. Source routing naturally supports any topology without per-router routing table storage, and allows the sender to choose among multiple paths.

HyNoC adopts source routing with a compact *relative* hop encoding: since a packet cannot exit from the same port it entered, a router with P ports offers only $P - 1$ egress choices, encoded in $\lceil \log_2(P - 1) \rceil$ bits per hop. This reduces both header overhead and crossbar complexity simultaneously.

C. Virtual Channels

Virtual channels (VCs), introduced by Dally [2], multiplex several logical channels over a single physical link, providing additional routing paths and breaking deadlock cycles. Their main drawback is area: each VC requires a dedicated buffer, and [3] shows that router area scales nearly linearly with VC count. On FPGAs, where block RAMs are shared between the application and the communication infrastructure, this overhead is particularly significant.

Several works have explored VC reduction or elimination. Turn-model routing [11] achieves deadlock freedom without VCs by restricting turns in the routing graph. HyNoC’s approach—providing path diversity through richer topologies and compile-time route selection—aligns with this trend, and is analyzed quantitatively in Section IV.

D. NoC Arbitration

Round-robin arbiters are widely used for their fairness guarantees. Sequential round-robin FSMs scan requests cyclically, introducing latency proportional to the number of ports. HyNoC implements a *parallel round-robin arbiter* (PRRA) that combines the fairness of round-robin with a low-latency response: the arbiter transitions directly to the state of any pending request, respecting a rotating priority order, and responds in one or two cycles regardless of the number of ports. The implementation relies on a LUT-mux structure that keeps the critical path independent of port count.

E. FPGA-Targeted NoCs

Several NoC designs target FPGAs specifically. CON-NECT [12] provides a parameterized, synthesizable NoC generator. HopliteRT [13] proposes a lightweight deflection-routing NoC without input buffers, achieving extremely small area at the cost of non-deterministic latency. HyNoC targets a different point: a fully parameterized, topology-agnostic NoC with bounded latency, suited to predictable VLIW workloads.

F. VLIW Many-Core Systems

VLIW architectures expose instruction-level parallelism explicitly, making them well suited for signal processing and scientific computing kernels [14]. Distributed VLIW many-core systems—exemplified by the Kalray MPPA [15]—replicate simple VLIW cores connected through a NoC. In such systems, communication patterns can be determined at compile time, making static source routing particularly attractive.

III. HYNO C ARCHITECTURE

A. Overview

A HyNoC network is built by assembling routers with a configurable number of ports (3 to 9 in the current implementation). Each router port is full-duplex, composed of an ingress interface and an egress interface. Two neighboring routers are connected by cross-linking their egress and ingress interfaces. Processing nodes are attached through a *local interface*, which adds an extra output FIFO to decouple the node’s clock domain from the router.

The router implements a **full crossbar**: any ingress port can simultaneously communicate with any egress port, subject to arbitration. Multiple paths can be open concurrently inside the same router, enabling full bisection bandwidth utilization when traffic permits. Figure 1 shows the internal organization of 5-port routers and their interconnections.

B. Ingress Port

The ingress port receives flits from the egress of a neighboring router. It buffers incoming flits in a dual-clock FIFO (2^D entries of $K + 1$ bits), which simultaneously handles clock domain crossing between the upstream router’s clock domain and the local router domain.

Once flits are buffered, the FSM inspects the two most significant bits of the first flit to identify the flit type (routing or payload). It then extracts the next-hop egress port identifier, asserts the matching *request* bit among the $N - 1$ available egress ports, and waits for the corresponding *grant*. Upon grant, the ingress forwards the routing flit updated to the next hop (index decremented); if the index has already reached zero, the routing flit is consumed locally and not forwarded. Subsequent payload flits are streamed directly to the granted egress until the stop bit of the last flit is detected, at which point the channel is released.

Flow control is managed by monitoring both the downstream router’s ingress FIFO level and the local ingress FIFO level. The ingress stalls when either FIFO approaches full, preventing flit loss. Flow control is managed by monitoring the FIFO level

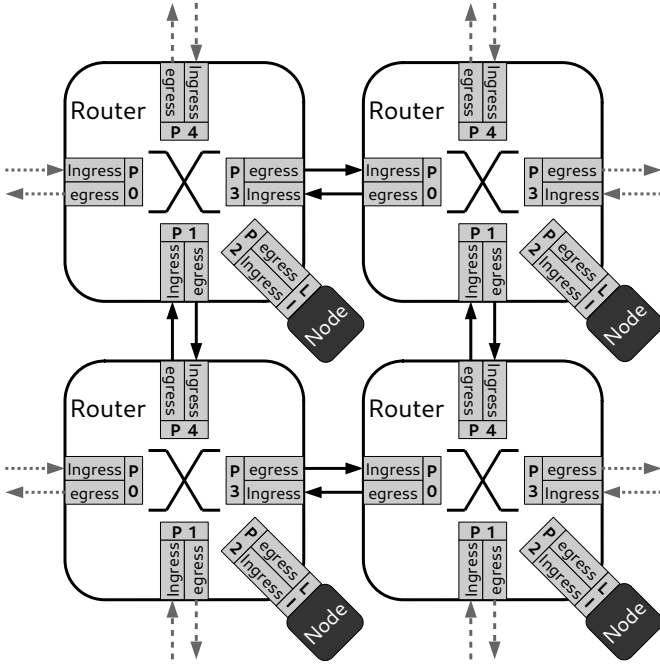


Fig. 1. Router interconnections overview.

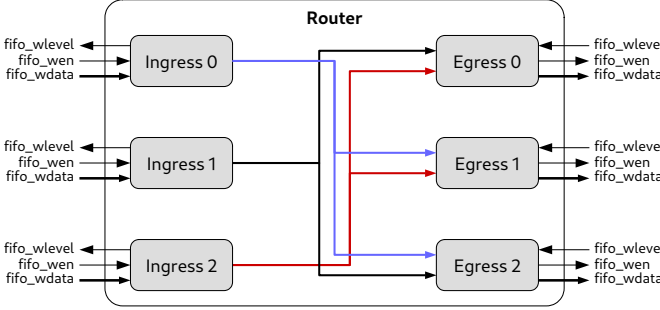


Fig. 2. Internal data path of a 3-port router.

of the downstream router's ingress FIFO. The ingress stalls when the downstream FIFO approaches full, preventing flit loss. The internal data path of a 3-port router is illustrated in Figure 2.

C. Egress Port

The egress port is connected to $N - 1$ ingress ports via a $(K + 1)$ -bit wide data path. It arbitrates among concurrent write requests using the PRRA, which responds in one cycle when the port is idle, and grants one ingress at a time without starvation. Once a grant is issued, the data and write-enable signals from the winning ingress are routed directly to the downstream router's ingress FIFO. The downstream FIFO level is forwarded back to the granted ingress for flow control. The control path is shown in Figure 4: contrary to the data path, control signals are point-to-point between each ingress and each egress, avoiding any broadcast of control information.

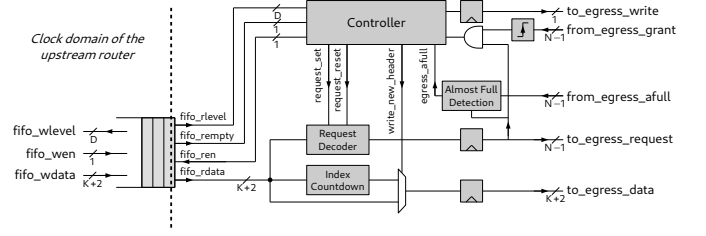


Fig. 3. Ingress port internal architecture: dual-clock FIFO and routing FSM.

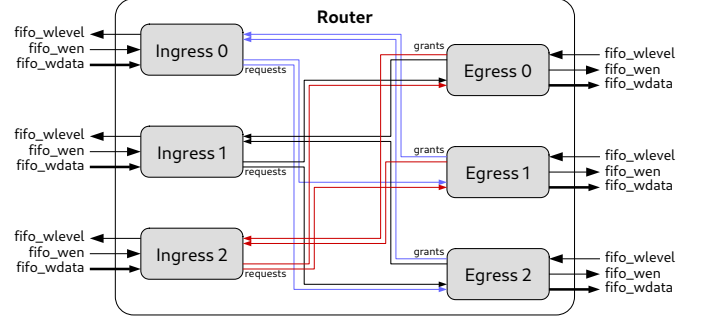


Fig. 4. Internal control path of a 3-port router.

D. Parallel Round-Robin Arbiter (PRRA)

The standard sequential round-robin FSM scans requests one by one in a fixed order, introducing up to N cycles of latency before granting a pending request (Figure 6). HyNoC replaces this with a **parallel round-robin arbiter (PRRA)**: a set of N combinatorial LUT-based state functions that evaluate all pending requests simultaneously and jump directly to the winning state, regardless of how many ports are idle.

The architecture instantiates N `prra_lut` modules in parallel (Figure 7), each pre-computing the next granted port for a different starting priority offset. The LUT corresponding to the current state is selected by a multiplexer; its output feeds the state register. The state transitions only when the currently granted request is de-asserted (end of packet) or when no request is active, locking the grant for the entire packet transfer and preventing spurious re-arbitration.

The arbiter responds in **one or two clock cycles** depending on the `PIPELINE` parameter: without pipelining the LUTs are purely combinatorial and the grant is issued in one cycle; with pipelining an extra register stage on the LUT outputs reduces the critical path at the cost of one additional cycle. The critical path does not grow with N because each LUT is independent and selected, not chained. Starvation is prevented by the rotating priority: after serving port k , priority rotates to port $k + 1 \bmod N$.

E. Clock Domain Architecture

Each router port operates in its own clock domain. The dual-clock FIFO at each ingress interface handles the crossing between the upstream port clock and the local router clock. For timing-critical designs, a single-clock mode

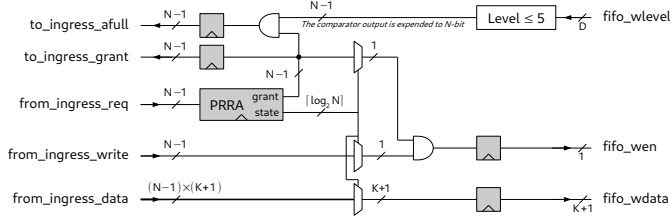


Fig. 5. Egress port internal architecture: PRRA arbiter and data multiplexer.

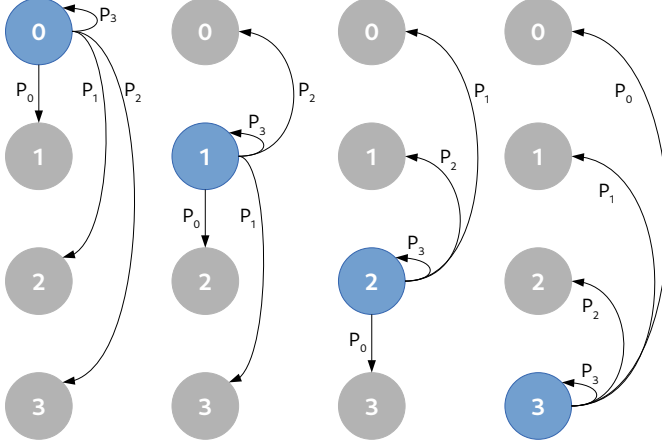


Fig. 6. Parallel Round-Robin FSM: direct state transitions avoid scanning idle ports.

(SINGLE_CLOCK_ROUTER=1) is available, reducing crossing latency.

F. Local Interface

Any router port can be connected to a processing node instead of a neighboring router. The `hynoc_local_interface` module handles this attachment by instantiating an additional FIFO at the egress output. This FIFO serves two purposes: it fully decouples the node's clock domain from the router clock domain, and it smooths traffic bursts, preventing the node from becoming a bottleneck when it cannot consume data fast enough. From the router's perspective, the local interface is indistinguishable from a downstream router's ingress port, preserving the uniformity of the flow control mechanism across all port types.

G. First Layer Protocol

1) *Packet and Flit Structure*: The network operates on flits of $K + 1$ bits ($K = \text{PAYLOAD_WIDTH}$). The most significant bit is a *stop bit*: when set, it marks the last flit of a packet. A packet is built with at least one *Network Hops flit* (routing flit) followed by at least one *payload flit*, as shown in Figure 8. If multiple routing flits are present, intermediate routers treat all but the first as payload; each router consumes one routing flit from the front of the chain. The MSB of a routing flit is always zero, reserving space for future sub-type extensions. Figure 9 shows the smallest valid packet.

The routing flit carries a 4-bit *Proto* field:

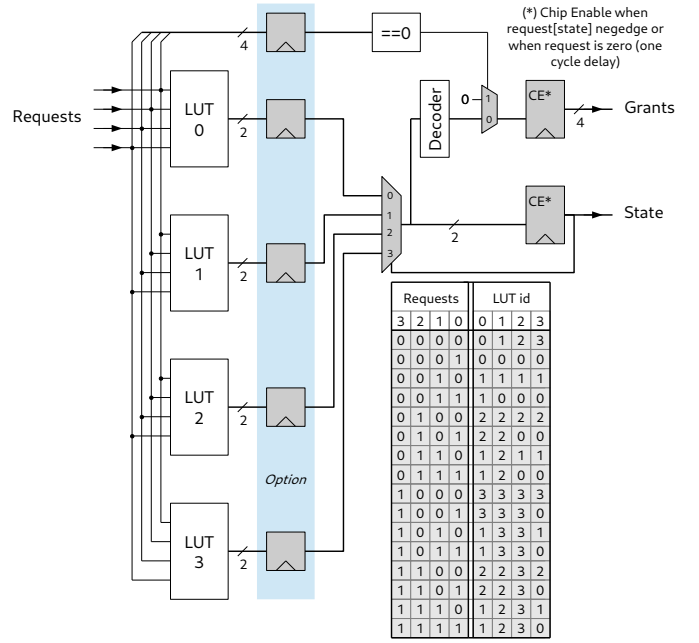


Fig. 7. PRRA architecture: N parallel LUTs, one per priority offset.

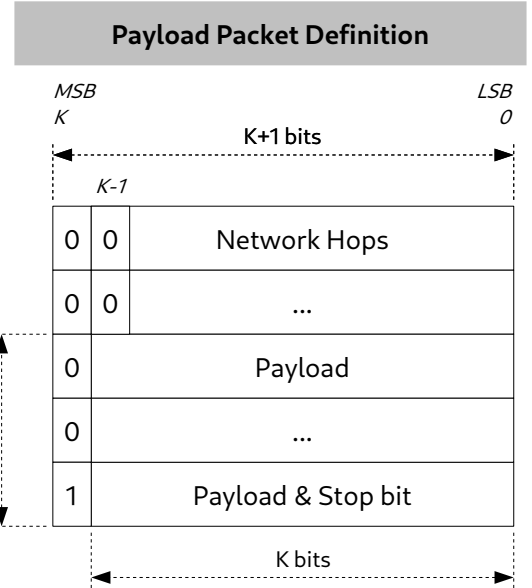


Fig. 8. Packet structure: Network Hops flit(s) followed by payload flits.

2) *Unicast Source Routing*: The Network Hops flit structure is shown in Figure 10. It carries H hop fields of $\lceil \log_2(P-1) \rceil$ bits each, encoding the relative egress port ID at the next router, together with an *index* field initialized to $H - 1$. The index directly feeds the multiplexer selecting the active hop in the ingress port. It is decremented after each hop is consumed; if the index is already zero before path opening, the routing flit is not forwarded downstream. The relative hop encoding is illustrated in Figure 11.

The relative encoding (egress IDs numbered counterclock-

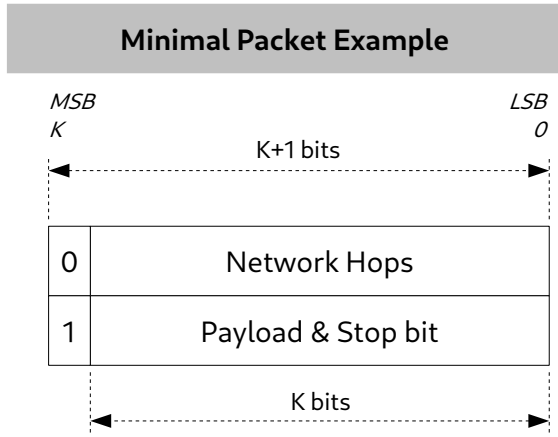


Fig. 9. Minimal HyNoC packet (one routing flit, one payload flit).

TABLE I
SUPPORTED ROUTING PROTOCOLS

Proto	Routing method
4'b0000	Unicast circuit switch
4'b0001	Multicast circuit switch
4'b1000	XY routing (reserved)
4'b1111	Forbidden

wise from the ingress port, excluding that port) reduces both the hop field width and the internal crossbar size. The relative encoding (egress IDs numbered counterclockwise from the ingress port, excluding that port) reduces both the hop field width and the internal crossbar size.

3) *Multicast Source Routing*: In multicast mode, each hop field is a bitmask of width $P - 1$, where each bit selects one egress port. The packet is duplicated and forwarded to all asserted egress ports simultaneously. Multiple routing flits with different multicast masks can be chained to address complex multicast trees. This is particularly useful for broadcast operations in VLIW many-core workloads.

IV. HYBRID SWITCHING MODEL AND DESIGN RATIONALE

A. The Hybrid Switching Model

HyNoC's switching model is best described as *circuit-switched wormhole*: it combines a **circuit-establishment phase** with a **wormhole data phase**.

In the **circuit-establishment phase**, the ingress port asserts a request to a specific egress port (identified from the routing flit) and waits for a grant. This request/grant handshake propagates along the route: once the first router grants the request, the ingress begins forwarding flits, which in turn trigger the same handshake at the next router. A dedicated forwarding path through the network is thus progressively opened, hop by hop.

In the **wormhole data phase**, payload flits are streamed flit-by-flit along the established path, regulated by FIFO-level flow control. The path remains dedicated to this packet until the stop bit is observed, ensuring in-order, contention-free delivery once the channel is open.

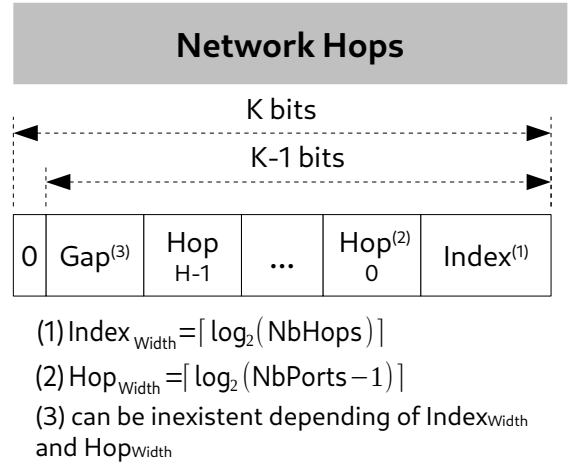


Fig. 10. Network Hops flit structure.

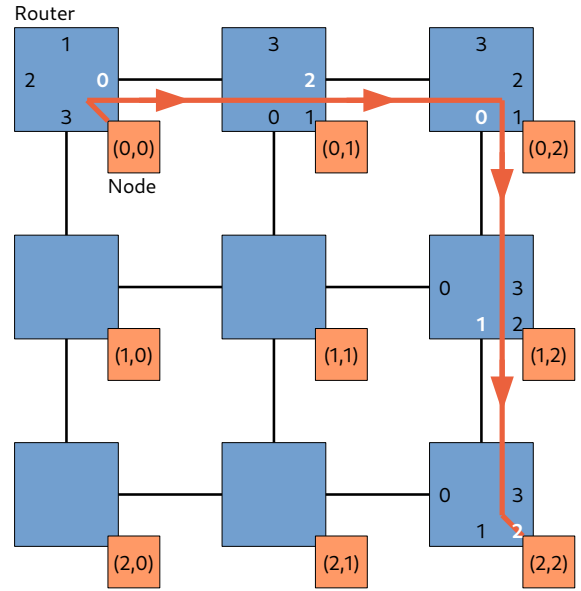


Fig. 11. Relative hop encoding for a 3×3 mesh network.

This combination provides the *low buffering* advantage of wormhole switching (only the ingress FIFO is needed, no full packet storage) while offering the *path dedication* advantage of circuit switching (no mid-network arbitration contention during payload transfer). The trade-off is a path-establishment latency proportional to the route length before the first payload flit is delivered at the destination.

B. Static Route Management as an Alternative to Virtual Channels

Virtual channels were introduced primarily to: (1) provide additional logical paths to avoid deadlock; and (2) reduce head-of-line blocking under congestion. HyNoC addresses both goals differently.

Deadlock avoidance is achieved through source routing itself: the complete path is determined by the sender and can

be constrained to acyclic subgraphs (e.g., dimension-order routing) without any router-internal mechanism.

Congestion avoidance is addressed at two levels. First, the compiler or runtime system assigns routes across multiple available shortest paths, distributing traffic away from known hotspots. The number of shortest paths between two nodes in an $N \times M$ mesh is a classical combinatorial result [16]: each path consists of a fixed sequence of moves along each dimension, taken in any order, and the count is given by the multinomial coefficient:

$$p = \binom{(N-1) + (M-1)}{N-1} = \frac{[(N-1) + (M-1)]!}{(N-1)!(M-1)!} \quad (1)$$

This generalizes to a K -dimensional network $n_1 \times n_2 \times \dots \times n_K$ as:

$$p = \frac{\left[\sum_{i=1}^K (n_i - 1) \right]!}{\prod_{i=1}^K (n_i - 1)!} \quad (2)$$

Note that (1) and (2) give the number of shortest paths between the *most distant* node pair (corner-to-corner). For an arbitrary pair separated by (d_x, d_y) hops the count is $\binom{d_x + d_y}{d_x}$, which is lower. Table II reports corner-to-corner values for representative 2D mesh topologies.

TABLE II
SHORTEST PATH COUNTS IN 2D MESH NOCs (CORNER-TO-CORNER)

Topology	Routers	Shortest paths	Path length
2×2	4	2	2
4×4	16	20	6
8×8	64	3 432	14
16×16	256	155 117 520	30

Second, for applications with more dynamic traffic, the network topology can be enriched with additional relay routers (without local interfaces), increasing path diversity proportionally to the added area.

C. Area Comparison with Virtual Channels

Following [3], router area scales linearly with the number of VCs (each VC requires an additional input buffer per port). An 8×8 mesh with 4 VCs consumes the equivalent area of a 16×16 mesh without VCs, yet the latter offers $66 \times$ more shortest paths between corner nodes. On FPGA, where block RAMs are scarce, minimizing buffer count is a primary concern.

The VC area model is itself optimistic in two respects: (1) the crossbar logic and VC scheduler also grow with VC count; (2) VC path diversity is an upper bound, since in practice the VC is assigned at the source and remains fixed for the entire path, providing less diversity than a true additional physical dimension. Conversely, the physical router area model does not capture link wiring cost, which may be relevant in ASIC contexts.

D. Head-of-Line Blocking and Mitigations

Once a channel is granted, it remains dedicated until the stop bit of the last flit is received. A long packet therefore delays other packets competing for the same egress port. Two complementary mitigations are available in HyNoC:

- 1) **Network enrichment**: adding more routers distributes traffic across more egress ports, reducing the probability that multiple flows compete for the same port. The cost is increased end-to-end *latency* due to additional hops—this is the primary performance trade-off, not throughput.
- 2) **Multiple local interfaces**: a node can expose several local interfaces connected to different router ports, providing as many independent logical channels as needed. This replicates the path diversity benefit of VCs without duplicating internal router buffers. It is the recommended approach when a node requires simultaneous, non-blocking communication with multiple peers.

E. Scope and Limitations

The topology-based approach is most effective when traffic patterns are known at compile time and routes can be assigned statically. When traffic is entirely unpredictable at design time and topology enrichment is not feasible, virtual channels remain the more appropriate solution. HyNoC targets systems—distributed VLIW processors on FPGA—where compile-time route assignment is natural and FPGA resource budgets make buffer minimization a primary concern.

V. TARGET APPLICATION: DISTRIBUTED VLIW COMPUTING

A. VLIW Cores and Compile-Time Scheduling

VLIW processors expose multiple functional units to the compiler, which schedules operations explicitly in long instruction words [14]. This design philosophy shifts control complexity from hardware to software, resulting in simple, area-efficient cores well suited for replication in many-core arrays. Signal processing, scientific computing, and neural network inference kernels are representative workloads.

In a distributed VLIW many-core system, each core executes a portion of a data-flow graph, passing intermediate results to neighboring cores through the NoC. The communication graph is typically known at compile time, enabling the compiler to:

- 1) Assign each communication a source route through the NoC;
- 2) Schedule packet injections to avoid simultaneous contention on the same egress port;
- 3) Guarantee bounded communication latency for critical paths.

This compile-time visibility makes HyNoC's hybrid model particularly effective: the circuit-establishment latency is paid once per packet and is deterministic (bounded by route length and PRRA grant latency), and the wormhole payload transfer proceeds without further arbitration overhead.

B. Integration with the Local Interface

Each VLIW core connects to the NoC through a local interface, which presents a simple FIFO-based write/read API. The core writes routing flits followed by payload flits into the local interface's ingress FIFO; the local interface manages all handshaking with the router. On the receive side, incoming flits are deposited in the egress FIFO and read by the core at its own pace. The dual-FIFO structure fully decouples the core's clock domain from the router domain.

The `hynoc_stream_writer` and `hynoc_stream_reader` modules are simulation utilities that generate and check random packet streams, respectively. They serve as traffic generators in the testbench environment, verifying the correctness of the NoC under various load conditions.

C. Multicast for Broadcast Operations

VLIW many-core applications frequently require broadcast operations: distributing a coefficient matrix, a lookup table, or a control token to multiple cores simultaneously. HyNoC's multicast routing protocol supports this natively: a single packet with a multicast routing flit is replicated at each router toward all targeted egress ports, without requiring the sender to issue multiple unicast packets. This reduces network load and sender overhead for one-to-many communication patterns common in data-parallel VLIW workloads.

VI. IMPLEMENTATION

A. RTL Organization

The HyNoC RTL is written in Verilog and organized as follows:

- `hynoc_ingress`: ingress port (dual-clock FIFO, FSM, routing protocol decoder for unicast and multicast);
- `hynoc_egress`: egress port (data multiplexer, PRRA);
- `hynoc_router_base`: parameterized router base (crossbar wiring, port instantiation);
- `hynoc_router_3p` / `hynoc_router_5p`: 3-port and 5-port router instances with self-checking testbenches;
- `hynoc_local_interface`: local interface for node attachment,
- `hynoc_stream_reader` / `hynoc_stream_writer`: simulation traffic generators used in testbenches to inject and check random packet streams. DMA-style packet injection and reception modules.

The design is parameterized by: payload width (`PAYLOAD_WIDTH`), FIFO depth (`LOG2_FIFO_DEPTH`: 2 to 64 entries), number of ports (`NB_PORTS`: 3 to 9), index width (`INDEX_WIDTH`), optional multicast routing (`ENABLE_MCAST_ROUTING`), and single-clock mode (`SINGLE_CLOCK_ROUTER`).

B. Simulation and Verification

Each module is delivered with a self-checking testbench. The 3-port router testbench exercises both unicast and multicast

scenarios over a 2-router, 4-node topology. The 5-port router testbenches cover five distinct traffic scenarios, including simultaneous multi-path transfers that verify the PRRA's starvation-free behavior.

Simulation is supported under Icarus Verilog, ModelSim, and Verilator. Waveform inspection is facilitated by Python-based waveform configuration scripts.

C. FPGA Target

Place-and-route results on an Intel Cyclone IV E FPGA are summarized in Table III. The router is configured with 32-bit payload and 16-entry dual-clock FIFOs per port. Logic grows approximately linearly with the number of ports, while the interface clock remains stable across configurations at 315 MHz.

TABLE III
SYNTHESIS RESULTS ON ALTERA CYCLONE IV E (32-BIT, 16-ENTRY FIFOs, SPEED-OPTIMIZED)

Router	LEs	Mem (bits)	FFs	f_{max} (MHz)
3-port	730	1600	450	280
5-port	1600	2650	750	245
7-port	2860	3700	1113	245
9-port	4250	4800	1500	230

The design targets Intel (Altera) and AMD (Xilinx) FPGA families. Reference board support is included for the DE0-Nano (Cyclone IV) and the ZedBoard (Zynq-7000). Dual-clock FIFOs are implemented using the `dclkfifolut` module, which maps to distributed LUT RAM, avoiding dependency on vendor-specific FIFO IP and enabling portability across FPGA families.

D. RTL Elaboration with Veriparse

HyNoC makes use of advanced Verilog constructs — in particular, `initial` blocks containing `for` loops and integer arithmetic — to pre-compute the PRRA lookup tables at elaboration time. While this is perfectly legal Verilog, some FPGA toolchains have limited support for such constructs in synthesizable RTL.

Veriparse [17] is a companion source-to-source transformation toolkit that addresses this issue. Its `veriflat` tool pre-processes a hierarchical design by flattening the module hierarchy, unrolling `generate` loops, constant-folding `localparam` expressions, and evaluating `initial` blocks into plain, widely-supported RTL. Applied to HyNoC, it produces a flat, fully unrolled Verilog file where each PRRA LUT entry is an explicit constant assignment — portable across any standard Verilog toolchain.

The flattened output also benefits simulation: a single-module design with no hierarchy reduces simulator overhead and can significantly speed up large testbench runs. Veriparse is available at <https://github.com/ccienti/veriparse> under the LGPLv3 license.

E. License

HyNoC is released as open-source hardware under the **CERN-OHL-P v2** license [18], a permissive open hardware license. The complete source, including RTL, testbenches, documentation, and build infrastructure, is available at: <https://github.com/cclienti/verilog-ip>.

VII. CONCLUSION

HyNoC is a hybrid circuit-switch/wormhole Network-on-Chip designed for distributed VLIW computing on FPGA. By combining source routing—which establishes a dedicated path through a distributed request/grant handshake—with wormhole flit streaming, HyNoC achieves bounded, deterministic transfer latency without the area overhead of virtual channels.

Congestion is managed statically by the compiler or system software, which selects routes across the rich set of shortest paths available in mesh and higher-dimensional topologies. The combinatorial analysis presented in Section IV shows that enriching the physical topology is more area-efficient than adding virtual channels for the targeted workload class, provided that a compile-time scheduler distributes traffic across available paths.

The parallel round-robin arbiter provides starvation-free, fixed-latency arbitration at each egress port, and per-port independent clock domains allow the NoC to interface cleanly with heterogeneous processing elements running at different frequencies. The multiple local interface mechanism offers a practical alternative to virtual channels when a node requires simultaneous non-blocking communication with multiple peers.

HyNoC is released as open-source hardware under the CERN-OHL-P v2 license, providing a complete RTL implementation with simulation testbenches and FPGA board support.

Future work includes: formal deadlock-freedom analysis for arbitrary source route sets; synthesis results and quantitative latency characterization on representative FPGA devices; integration with a VLIW compiler back-end for automatic route assignment and injection scheduling; and extension of the XY routing protocol placeholder to a full hardware implementation.

REFERENCES

- [1] W. J. Dally and B. Towles, “Route packets, not wires: On-chip interconnection networks,” in *Proc. Design Automation Conference (DAC)*, 2001, pp. 684–689.
- [2] W. J. Dally, “Virtual-channel flow control,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 3, no. 2, pp. 194–205, 1992.
- [3] A. Mello, L. Tedesco, N. Calazans, and F. Moraes, “Virtual channels in networks on chip: Implementation and evaluation on Hermes NoC,” in *Proc. 18th Annual Symposium on Integrated Circuits and System Design (SBCCI)*, 2005, pp. 178–183.
- [4] F. Moraes, N. Calazans, A. Mello, L. Möller, and L. Ost, “HERMES: an infrastructure for low area overhead packet-switching networks on chip,” *Integration, the VLSI Journal*, vol. 38, no. 1, pp. 69–93, 2004.
- [5] W. J. Dally and C. L. Seitz, “Deadlock-free message routing in multiprocessor interconnection networks,” vol. 36, no. 5, 1987, pp. 547–553.
- [6] S. Kumar *et al.*, “A network on chip architecture and design methodology,” in *Proc. IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2002, pp. 105–112.
- [7] K. Goossens, J. Dielissen, and A. Radulescu, “Ethereal network on chip: Concepts, architectures, and implementations,” *IEEE Design & Test of Computers*, vol. 22, no. 5, pp. 414–421, 2005.
- [8] J. Duato, “A new theory of deadlock-free adaptive routing in wormhole networks,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 4, no. 12, pp. 1320–1331, 1993.
- [9] M. Liwei and S. Yihe, “On-chip network design automation with source routing switches,” vol. 12, no. 1, 2007, pp. 77–85.
- [10] S. Mubeen and S. Kumar, “Designing efficient source routing for mesh topology network on chip platforms,” in *Proc. 13th Euromicro Conf. on Digital System Design (DSD)*, 2010, pp. 181–188.
- [11] C. J. Glass and L. M. Ni, “The turn model for adaptive routing,” in *Proc. International Symposium on Computer Architecture (ISCA)*, 1992, pp. 278–287.
- [12] M. K. Papamichael and J. C. Hoe, “CONNECT: Re-examining conventional wisdom for designing NoCs in the context of FPGAs,” in *Proc. ACM/SIGDA International Symposium on FPGAs (FPGA)*, 2012, pp. 37–46.
- [13] S. M. Nair *et al.*, “HopliteRT: A real-time aware NoC for FPGAs,” in *Proc. International Conference on Field Programmable Logic and Applications (FPL)*, 2019.
- [14] J. R. Ellis, “Bulldog: A compiler for VLIW architectures,” Ph.D. dissertation, Yale University, 1986.
- [15] B. de Dinechin *et al.*, “A clustered manycore processor architecture for embedded and accelerated applications,” in *Proc. IEEE High Performance Extreme Computing (HPEC)*, 2013, pp. 1–6.
- [16] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997, section 1.2.5: Permutations and Factorials.
- [17] C. Clienti, “Veriparse: A source-to-source transformation toolkit for verilog/systemverilog,” <https://github.com/cclienti/veriparse>, 2024, IGPLv3 license.
- [18] “CERN open hardware licence version 2 – permissive,” https://ohwr.org/cern_ohl_p_v2.txt, 2020.